

End-to-End Python Pipeline for a Multilingual South African Institutional Semantic-Validity Dataset

Executive summary

You can build a research-grade **institutional semantic-validity** dataset for mechanistic interpretability by extracting self-contained procedural sentences from **Autshumato** (formal, aligned, government-domain) and **Vuk'uzenzele** (public-facing government publication with multi-language PDFs/translations), then generating **minimal invalid perturbations** and **base/local pairs** for activation patching—while enforcing stringent leakage controls. Autshumato's official data page explicitly describes its **parallel corpora as sentence-level collections aligned with English**, created via automatic + manual alignment and **mainly sourced from the South African government domain**. ¹ Vuk'uzenzele's official archive and edition pages provide **PDFs and "Translations" links across multiple official languages**, and GCIS states it is published **twice a month**. ²

The core research risk is "task collapse": without careful controls, models can solve the dataset using shortcuts such as (i) **source style**, (ii) **obvious weird-word detection**, (iii) **named-entity heuristics**, or (iv) **format artifacts** from PDF extraction—none of which supports mechanistic claims about localisation. Your pipeline must therefore be judged by (a) **operational filtering rigor**, (b) family-based splitting, and (c) **weak-baseline leakage probes** (source-only, n-gram, and style-feature predictors) with explicit pass/fail thresholds.

This report provides an implementable design with Python 3.10+ code snippets for all key components (ingest → segment → normalize → lexicon/relation checks → self-containment → dedup → perturb → pair-build → balance/split → QC probes → export) and language-aware fallback heuristics for cases where dependency parsers are unavailable. It assumes you will run locally and that corpora are already available on disk; code avoids any downloading.

flowchart TD

```
A[Local corpora on disk] --> B[Ingest: Autshumato + Vuk'uzenzele PDFs]
B --> C[Normalize text: UTF-8/Unicode + PDF cleanup]
C --> D[Sentence segmentation per language profile]
D --> E[Candidate mining: lexicons + relation checks]
E --> F[Filtering: metadata removal + self-containment + stability]
F --> G[Dedup: exact + MinHash LSH + embedding]
G --> H[Family typing: institution/service/document/locality]
H --> I[Perturbations: typed swaps + minimality scoring]
I --> J[Base/Local pair-build for patching]
J --> K[Balancing + group-aware stratified split]
```

```
K --> L[QC: leakage probes + duplicate audits]
L --> M[Export exact schema CSV]
M --> N[Pilot + gold validation sampling + human review]
```

Data sources, languages, and compliance constraints

South Africa has multilingual obligations in national government communication; one government policy document explicitly lists the 11 official languages (Sepedi, Sesotho, Setswana, siSwati, Tshivenda, Xitsonga, Afrikaans, English, isiNdebele, isiXhosa, isiZulu). ³ (South African Sign Language has since been approved as a 12th official language; include it only if your pipeline and corpus sources support it in practice.) ⁴

Autshumato is a foundational source for your use case because it provides bilingual aligned data between English and all other official written South African languages, plus monolingual data; it was sourced from translations/publications and government web crawling, and intended for MT but reusable for other NLP tasks. ⁵ SADIaR metadata for specific language-pair releases records that the corpora are **aligned** and **sentence segmented**, and includes licensing fields (for example, CC BY-NC-SA 2.5 ZA on the English-Afrikaans release). ⁶ The SADIaR record for an English-isiZulu aligned corpus also describes distribution as two UTF-8 text files with newline-aligned segments and notes that further cleanup may be required depending on reuse. ⁷

Vuk'uzenzele provides a different register: it is a GCIS publication produced twice monthly, and official edition pages often list “English PDF version” and “Translations” into multiple official languages (e.g., Afrikaans, isiNdebele, isiXhosa, isiZulu, Sepedi, Sesotho, Setswana). ⁸ The official site’s archives page provides browsing by edition. ⁹

A critical compliance caveat: Vuk'uzenzele pages display “Copyright © 2026 All rights reserved,” which may constrain public redistribution of extracted text (even if use for research is fine). ⁹ If you intend to publish the dataset, plan early for a redaction/ID-only strategy or to rely on a separately licensed derived corpus (and document provenance and permissions explicitly).

Operational filtering specification with language classes and fallback heuristics

Your filtering spec must be strong enough to keep candidates that are **institutionally grounded and self-contained** and to exclude anything where validity depends on missing context, formatting, or temporally brittle details.

Language classes and parsing strategy

Because dependency-parsing support varies by language, treat languages as profiles:

Language profile	Languages (recommended ISO-639-3)	Parser available?	Relation check strategy	Segmentation strategy
English profile	eng	Yes (spaCy) ¹⁰	Dependency patterns + lemma match	spaCy pipeline sentence boundaries; fallback to Sentencizer ¹¹
Afrikaans UD profile	afr	Sometimes (UD resources exist; Stanza may support)	Prefer UD dependency patterns when parser available; else window+frames	Stanza if available; else Sentencizer+regex
Bantu fallback profile	zul,xho,nbl,ssw,nso,sot,tsn,ven,tso	Often not in mainstream toolchains	Window co-occurrence + curated procedural frames	Sentencizer+regex; extra caution for PDF headers/line breaks

Afrikaans has a UD treebank (“UD Afrikaans-AfriBooms”) described as consisting of public government documents, which makes it domain-appropriate for procedural parsing heuristics.¹² However, you should implement “parser availability” as a runtime capability (attempt to load; otherwise fall back), because tool support changes.

Normalization requirements

For PDF-based sources (Vuk’uzenzele), use `pdfplumber` as a default; it “works best on machine-generated, rather than scanned, PDFs” and is built on `pdfminer.six`.¹³ Use `pdfminer.six` as a fallback when you need lower-level layout handling; it focuses on extracting/analyzing text and can expose location/font information.¹⁴

Hard exclusions (regex + numeric/style filters)

Apply these before deep parsing to eliminate non-propositional strings and boilerplate:

- URLs: `https?://\S+|www\.\S+`
- Emails: `\b[\w.\- ]+@[\w.\- ]+\.\w+\b`
- Phone-like patterns (conservative)
- Bullet/heading patterns: leading `•`, `-`, `*`, or numbered headings (`^\s*\d+[\.\. ]\s+`)
- High “metadata density”: digit ratio > 0.25 OR uppercase ratio > 0.85 AND token count < 12
- Drop if contains obvious archive/footer artifacts (e.g., “PAIA Manual”, “Privacy Policy”) commonly present in website page headers/footers on Vuk’uzenzele.⁹

Declarative single-sentence rule

Keep only if all hold:

- exactly one sentence after segmentation
- excludes questions/exclamations (?, !)
- no multi-sentence paragraphs (drop if internal sentence boundary detected)

Length thresholds: whitespace + subword

Use dual thresholds to normalize across agglutinative morphology:

- whitespace tokens:
- English/Afrikaans: 8–30
- Bantu languages: 6–28 (tune after pilot)
- SentencePiece pieces: 12–64 for all languages (SentencePiece supports direct training from raw sentences and avoids language-specific pre/post-processing). ¹⁵

Institutional grounding: lexicon + relation checks

Require:

- ≥ 1 institution lexicon match
- ≥ 1 action/process lexicon match
- relation check passes (parser-based where available; window+frames otherwise)

spaCy Sentencizer exists precisely for cases where you want rule-based sentence boundary detection without dependency parsing. ¹¹ Use it for low-resource language profiles and for corrupted PDFs.

English dependency relation patterns (spaCy)

Using spaCy dependency parses, accept if any:

- institution mention is subject/object of an action verb whose lemma is in action lexicon
- institution mention is in a prepositional phrase that is attached to an action verb

Afrikaans (UD-style) dependency patterns (if parsed)

If you have UD parses, accept patterns analogous to English using UD labels such as `nsubj`, `obj`, `obl`, `case`.

Bantu fallback relation checks

Use:

- window co-occurrence: institution and action terms within ± 8 whitespace tokens
- curated procedural frame patterns per family (Nguni vs Sotho-Tswana vs Tsonga/Venda):

- modal/obligation frames (“must/should/need to” equivalents) + apply/register/submit equivalents + institution anchor in same clause window

Self-containment scoring (0–3; keep ≥ 2)

Operational score:

- +1 if sentence does not start with a pure anaphor (language-specific list) unless an explicit antecedent exists in first ~8 tokens
- +1 if contains an argument anchor (applicant/patient/learner/resident/beneficiary; language-specific)
- +1 if contains an access/locative frame (“at/at the office/clinic/school...”, or language equivalents)

Temporal brittleness exclusion

Drop sentences where validity depends on time-sensitive policy facts unless you preserve context (which you explicitly want to avoid):

- contains hard dates or “edition”-specific deadlines (e.g., “applications close on ...”)
- relies on “this year/now/currently” without a general procedural clause

Implementable pipeline architecture and runnable code skeleton

This section gives you module responsibilities, data structures, and concrete Python 3.10+ code snippets for all key steps. The code is intentionally “runnable scaffolding”: it will execute once you supply local paths and lexicons, but it will require you to tune lexicons and thresholds with your pilot review.

Recommended environment and Docker setup

Use Python 3.10+. Keep intermediate artifacts in Parquet for speed; PyArrow enables reading/writing Parquet and integrates with pandas. ¹⁶

Minimum library set and why:

- `pdfplumber` for PDFs ¹⁷
- `pdfminer.six` fallback / layout ¹⁸
- `spacy` (+ `en_core_web_sm`) for English parsing and/or Sentencizer ¹¹
- `stanza` optional for UD pipelines; Stanza uses UD-based models and documents UD v2.12 usage for tokenization/POS/lemma/depparse in its available models page ¹⁹
- `sacremoses` for normalization/tokenization utilities where appropriate (e.g., consistent punctuation normalization across languages) ²⁰
- `sentencepiece` for subword length control ¹⁵
- `sentence-transformers` for embedding similarity/dedup/minimality scoring ²¹
- `datasketch` for MinHash LSH near-dup ²²
- `RapidFuzz` for fast edit-distance ratios ²³
- `scikit-learn` for splits + leakage probes (`StratifiedGroupKFold`, `LogisticRegression`) ²⁴

- `fasttext` language ID model `lid.176` as a QA gate (176 languages, confidence scores) ²⁵
- `krippendorff` for alpha reporting ²⁶
- `doccano` (optional) for annotation UI ²⁷

File layout and staged artifacts

Store rich internal metadata during processing, then export only the required schema:

Artifact	Format	Purpose
<code>interim/sentences.parquet</code>	Parquet	One row per sentence segment with provenance + features
<code>interim/candidates.parquet</code>	Parquet	Filtered base candidates
<code>interim/families.parquet</code>	Parquet	Families + typed fields used for perturb/pair
<code>interim/examples.parquet</code>	Parquet	Final examples with hidden metadata (source, family_id, perturb_type)
<code>final/dataset.csv</code>	CSV	Export exact schema only
<code>final/pilot_240.csv</code>	CSV	240-row inspection pilot (exact schema + optional separate metadata file)
<code>final/gold200.csv</code>	CSV	200-sample gold validation set for human review (exact schema)

Core configuration: language profiles

Create a `language_profiles.py` so you don't hardcode language behavior.

```
# src/language_profiles.py
from dataclasses import dataclass
from typing import Literal, Optional

RelationMode = Literal["spacy_dep", "ud_dep", "window_frames"]
SegmentMode = Literal["spacy_sents", "stanza_sents", "sentencizer_regex"]

@dataclass(frozen=True)
class LanguageProfile:
    iso3: str
    name: str
    segment_mode: SegmentMode
    relation_mode: RelationMode
    ws_min: int
    ws_max: int
    sp_min: int
```

```

    sp_max: int

PROFILES = {
    "eng": LanguageProfile("eng", "English", "spacy_sents", "spacy_dep", 8, 30, 12,
64),
    "afr": LanguageProfile("afr", "Afrikaans", "stanza_sents", "ud_dep", 8, 30, 12,
64),
    # Bantu fallback defaults
    "zul": LanguageProfile("zul", "isiZulu", "sentencizer_regex", "window_frames",
6, 28, 12, 64),
    "xho":
LanguageProfile("xho", "isiXhosa", "sentencizer_regex", "window_frames", 6, 28, 12,
64),
    "nbl":
LanguageProfile("nbl", "isiNdebele", "sentencizer_regex", "window_frames", 6, 28,
12, 64),
    "ssw": LanguageProfile("ssw", "siSwati", "sentencizer_regex", "window_frames",
6, 28, 12, 64),
    "nso": LanguageProfile("nso", "Sepedi", "sentencizer_regex", "window_frames",
6, 28, 12, 64),
    "sot": LanguageProfile("sot", "Sesotho", "sentencizer_regex", "window_frames",
6, 28, 12, 64),
    "tsn":
LanguageProfile("tsn", "Setswana", "sentencizer_regex", "window_frames", 6, 28, 12,
64),
    "ven":
LanguageProfile("ven", "Tshivenda", "sentencizer_regex", "window_frames", 6, 28, 12,
64),
    "tso":
LanguageProfile("tso", "Xitsonga", "sentencizer_regex", "window_frames", 6, 28, 12,
64),
}

```

Ingest: Autshumato

Autshumato aligned corpora are described as sentence-level aligned with English. ²⁸ Your ingest should read local aligned files and emit “sentence rows” with provenance.

```

# src/ingest_autshumato.py
from pathlib import Path
from typing import Iterable
import pandas as pd

def ingest_autshumato(en_file: Path, xx_file: Path, xx_iso3: str, source:
str="autshumato") -> pd.DataFrame:
    en_lines = en_file.read_text(encoding="utf-8").splitlines()

```

```

xx_lines = xx_file.read_text(encoding="utf-8").splitlines()
if len(en_lines) != len(xx_lines):
    raise ValueError(f"Alignment mismatch: {len(en_lines)} vs
{len(xx_lines)}")

rows = []
for i, (en, xx) in enumerate(zip(en_lines, xx_lines)):
    en = en.strip()
    xx = xx.strip()
    if not en or not xx:
        continue
    rows.append({
        "source": source,
        "doc_id": f"{en_file.stem}__{xx_iso3}",
        "sent_id": f"{en_file.stem}__{xx_iso3}__{i}",
        "language": "eng",
        "text_raw": en,
    })
    rows.append({
        "source": source,
        "doc_id": f"{en_file.stem}__{xx_iso3}",
        "sent_id": f"{en_file.stem}__{xx_iso3}__{i}",
        "language": xx_iso3,
        "text_raw": xx,
    })
return pd.DataFrame(rows)

```

Ingest: Vuk'uzenzele PDFs

Vuk'uzenzele offers PDFs and translations at edition pages and archives. ²⁹ Use `pdfplumber` for machine-generated PDFs; it is built on `pdfminer.six`. ¹³

```

# src/ingest_vukuzenzele.py
from pathlib import Path
import pandas as pd
import pdfplumber

def extract_pdf_text(pdf_path: Path) -> str:
    parts = []
    with pdfplumber.open(str(pdf_path)) as pdf:
        for page in pdf.pages:
            parts.append(page.extract_text() or "")
    return "\n".join(parts)

def ingest_vukuzenzele_pdf(pdf_path: Path, lang_iso3: str, edition_id: str,
source: str="vukuzenzele") -> pd.DataFrame:

```

```

raw = extract_pdf_text(pdf_path)
return pd.DataFrame([
    "source": source,
    "doc_id": f"{edition_id}__{lang_iso3}",
    "sent_id": f"{edition_id}__{lang_iso3}__doc",
    "language": lang_iso3,
    "text_raw": raw,
})

```

Normalize

```

# src/normalize.py
import re
import ftty

_WS = re.compile(r"\s+")
_DEHYPH = re.compile(r"(\w)-\s*\n\s*(\w)")

def normalize_text(text: str) -> str:
    text = ftty.fix_text(text)
    text = _DEHYPH.sub(r"\1\2", text)
    text = text.replace("\u00ad", "")
    text = text.replace("\r", "\n")
    text = _WS.sub(" ", text).strip()
    return text

```

Segment

For low-resource languages, use spaCy's Sentencizer (rule-based, no dependency parse). ¹¹

```

# src/segment.py
import re
import spacy

_SENT_SPLIT_FALLBACK = re.compile(r"(?<=[.!?])\s+")

def make_spacy_en():
    return spacy.load("en_core_web_sm", exclude=["ner"])

def make_sentencizer():
    nlp = spacy.blank("xx")
    nlp.add_pipe("sentencizer")
    return nlp

def segment_text(text_norm: str, lang_iso3: str, nlp_en=None, nlp_sent=None) ->

```

```

list[str]:
    if lang_iso3 == "eng":
        doc = nlp_en(text_norm)
        return [s.text.strip() for s in doc.sents if s.text.strip()]

    # fallback: sentencizer + regex fallback
    doc = nlp_sent(text_norm)
    sents = [s.text.strip() for s in doc.sents if s.text.strip()]
    if len(sents) <= 1:
        sents = [s.strip() for s in _SENT_SPLIT_FALLBACK.split(text_norm) if
s.strip()]
    return sents

```

Lexicon match

```

# src/lexicon_match.py
from typing import Dict, List

def lexicon_match(text_norm: str, lex: Dict[str, str]) -> List[str]:
    low = text_norm.casefold()
    hits = []
    for surface, tag in lex.items():
        if surface in low:
            hits.append(tag)
    return hits

```

Relation checks: dependency or window/frames

```

# src/relation_check.py
from typing import Set

def relation_window(tokens: list[str], inst_terms: Set[str], act_terms:
Set[str], window: int=8) -> bool:
    inst_pos = [i for i,t in enumerate(tokens) if t in inst_terms]
    act_pos = [i for i,t in enumerate(tokens) if t in act_terms]
    for i in inst_pos:
        for j in act_pos:
            if abs(i-j) <= window:
                return True
    return False

def relation_spacy_en(doc, inst_surfaces: set[str], act_lemmas: set[str]) ->
bool:
    inst_tokens = [t for t in doc if t.text.casefold() in inst_surfaces]
    act_tokens = [t for t in doc if t.pos_ == "VERB" and t.lemma_.casefold()

```

```

in act_lemmas]
    if not inst_tokens or not act_tokens:
        return False

    for inst in inst_tokens:
        if inst.dep_ in {"nsubj", "nsubjpass", "obj", "iobj", "pobj"}:
            if inst.head in act_tokens or inst.head.head in act_tokens:
                return True
    return False

```

Filtering and self-containment

```

# src/filter_candidates.py
import re

URL_RE = re.compile(r"https?://\S+|www\.\S+", re.I)
EMAIL_RE = re.compile(r"\b[\w.\-]+@[ \w.\-]+\.\w+\b")
BULLET_RE = re.compile(r"^\s*([\-\*\u2022]|\d+[\.\.])\s+")

def is_metadata_like(text: str) -> bool:
    if URL_RE.search(text) or EMAIL_RE.search(text):
        return True
    if BULLET_RE.search(text):
        return True
    digits = sum(ch.isdigit() for ch in text)
    uppers = sum(ch.isupper() for ch in text)
    n = max(len(text), 1)
    digit_ratio = digits / n
    upper_ratio = uppers / n
    if digit_ratio > 0.25:
        return True
    if upper_ratio > 0.85 and len(text.split()) < 12:
        return True
    return False

ANAPHOR_START = {
    "eng": re.compile(r"^(this|that|these|those|it|they|he|she|we)\b", re.I),
    "afr": re.compile(r"^(dit|daardie|hierdie|hulle|hy|sy|ons)\b", re.I),
    # Extend per language after pilot
}

def self_containment_score(text: str, lang: str, arg_terms: set[str],
access_terms: set[str]) -> int:
    score = 0
    words = text.split()
    if words:

```

```

    rx = ANAPHOR_START.get(lang)
    if (rx is None) or (not rx.match(words[0])):
        score += 1
    low = text.casefold()
    if any(t in low for t in arg_terms):
        score += 1
    if any(t in low for t in access_terms):
        score += 1
    return min(score, 3)

```

Dedup: exact + MinHash LSH + embedding

Use MinHash LSH from datasketch for scalable near-duplicate detection. ²² Use Sentence-Transformers for embedding similarity. ²¹ RapidFuzz provides fast string similarity and is MIT-licensed. ²³

```

# src/dedup.py
import hashlib
from datasketch import MinHash, MinHashLSH

def exact_hash(text_norm: str) -> str:
    return hashlib.sha1(text_norm.encode("utf-8")).hexdigest()

def build_minhash(tokens: set[str], num_perm: int=128) -> MinHash:
    mh = MinHash(num_perm=num_perm)
    for t in tokens:
        mh.update(t.encode("utf-8"))
    return mh

def lsh_dedup(records: list[dict], text_key: str="text_norm", threshold:
float=0.9) -> list[dict]:
    # records must have stable 'id' field
    lsh = MinHashLSH(threshold=threshold, num_perm=128)
    sigs = {}
    for r in records:
        toks = set(r[text_key].casefold().split())
        mh = build_minhash(toks)
        sigs[r["id"]] = mh
        lsh.insert(r["id"], mh)

    kept = []
    seen = set()
    for r in records:
        if r["id"] in seen:
            continue
        nbrs = lsh.query(sigs[r["id"]])
        # keep the first, mark others as duplicates

```

```

        kept.append(r)
    for n in nbrs:
        seen.add(n)
    return kept

```

Embedding dedup (pairwise at scale requires batching + ANN; for pilot-size you can do simple cosine thresholding):

```

# src/dedup_embeddings.py
import numpy as np
from sentence_transformers import SentenceTransformer

def embedding_dedup(texts: list[str], ids: list[str], model_name: str, cos_thr:
float=0.95) -> list[str]:
    model = SentenceTransformer(model_name)
    emb = model.encode(texts, normalize_embeddings=True, batch_size=64,
show_progress_bar=True)
    keep = []
    dropped = set()
    for i, id_i in enumerate(ids):
        if id_i in dropped:
            continue
        keep.append(id_i)
        sims = emb @ emb[i]
        for j, s in enumerate(sims):
            if j != i and s >= cos_thr:
                dropped.add(ids[j])
    return keep

```

Lexicon design and multilingual mapping

Lexicons are the “spine” of this task: they drive extraction filters, perturbation swaps, pairing, and balancing. They must be typed, multilingual, and auditable.

Typed lexicon schema

Maintain YAML/JSON entries with:

- `surface`: string (lowercased)
- `lang`: ISO-639-3
- `kind`: institution | action | document | locality | argument | access_frame
- `type`: semantic type for compatibility matrix (e.g., `identity_docs`, `social_grants`)
- `local_marker`: boolean (for locality cues)

Example entries (illustrative; you should corpus-attest):

surface	lang	kind	type	local_marker
"home affairs"	eng	institution	identity_docs	1
"sassa"	eng	institution	social_grants	1
"thusong service centre"	eng	institution	service_hub	1
"apply"	eng	action	apply	0
"faka isicelo"	zul	action	apply	0
"eGauteng"	zul	locality	province	1

For locality anchors, you can include official SA service-hub names such as Thusong Service Centres (directory listings exist on the SA government portal). ³⁰

Extending lexicons

Use a three-tier strategy to expand safely:

- **Autshumato alignment projection (preferred):** Autshumato provides bilingual aligned data between English and other SA languages, enabling lexicon induction from aligned segments. ⁵
Operational approach: for each English seed term, find aligned target-side n-grams with high co-occurrence and PMI; shortlist; require target-side attestation frequency.

- **Terminology lists / bilingual dictionaries (high precision, but must be attested):** South African government-hosted multilingual terminology PDFs can seed candidates; for example, DSAC hosts multilingual terminology lists spanning multiple official languages. ³¹
Rule: never activate a term solely because it appears in a terminology list—require corpus attestation in Autshumato or Vuk'uzenzele text.

- **NLLB-assisted translation (candidate generation only):** NLLB is documented as a model that supports translation across 200+ languages; Hugging Face Transformers docs note that generation should set `forced_bos_token_id` to the target language id and uses FLORES/BCP-47 tags like `fra_Latn`. ³²
Verification rules:

- Treat NLLB outputs only as candidates.
- Require corpus attestation OR bilingual validator confirmation.
- Require that the candidate preserves register and doesn't introduce rare/archaic morphology.

Mapping notes: NLLB commonly uses tags like `eng_Latn`, `zul_Latn`. Keep an internal mapping table ISO-639-3 → NLLB code. The official Transformers documentation describes BCP-47/FLORES tags and decoding workflow. ³³

Perturbation rules, minimality scoring thresholds, and pairing logic

Perturbation types and selection

Your invalid examples must be “near misses” that are grammatically plausible but institutionally wrong. Use typed swaps that violate exactly one compatibility constraint.

Perturbation type	Swap target	Compatibility violated	Typical strength
Institution swap	institution phrase	institution→service mismatch	high
Service swap	service phrase	institution→service mismatch	high
Document swap	document phrase	service→document mismatch	medium-high
Role swap	agent role	role→action mismatch	medium
Locality mismatch	local system/place	locality→infrastructure mismatch	high but risky

Selection algorithm:

1. Parse/identify anchor spans via lexicon matches.
2. Generate up to `k=5` candidates across perturb types (priority: institution → service → document → role → locality).
3. For each candidate compute minimality scores; discard those outside thresholds.
4. Choose highest semantic-similarity candidate that still violates the compatibility matrix.

Minimality scoring: operational metrics and thresholds

Use three metrics:

- **RapidFuzz similarity ratio:** RapidFuzz is a fast string matching library and is MIT licensed. ²³
- **Embedding cosine similarity:** Sentence-Transformers provides a Python module for embedding similarity/scoring. ²¹
- **Token edit count:** count changed tokens (whitespace tokens or aligned spans)

Recommended initial thresholds (tune on pilot):

- RapidFuzz ratio ≥ 0.88
- embedding cosine ≥ 0.82
- token edits ≤ 4

Runnable perturbation scaffold

```
# src/perturb.py
from dataclasses import dataclass
from typing import Optional, Dict, List, Tuple
from rapidfuzz.fuzz import ratio as rf_ratio
```

```

from sentence_transformers import SentenceTransformer
import numpy as np

@dataclass(frozen=True)
class SwapCandidate:
    pert_type: str
    text: str
    rf: float
    cos: float
    edits: int
    ok: bool

def token_edit_count(a: str, b: str) -> int:
    ta, tb = a.split(), b.split()
    # cheap proxy: count positions with different token, plus length delta
    n = min(len(ta), len(tb))
    diffs = sum(1 for i in range(n) if ta[i] != tb[i])
    diffs += abs(len(ta) - len(tb))
    return diffs

def cosine_sim(embed_a: np.ndarray, embed_b: np.ndarray) -> float:
    # embeddings assumed normalized
    return float(np.dot(embed_a, embed_b))

def generate_institution_swap(base: str, inst_surface: str, repl_surface: str) -> str:
    # minimal, surface-level replacement
    return base.replace(inst_surface, repl_surface)

def score_candidate(base: str, cand: str, model: SentenceTransformer,
                    rf_thr=88.0, cos_thr=0.82, edit_thr=4) ->
    Tuple[float, float, int, bool]:
    rf = rf_ratio(base, cand)
    emb = model.encode([base, cand], normalize_embeddings=True)
    cos = cosine_sim(emb[0], emb[1])
    edits = token_edit_count(base, cand)
    ok = (rf >= rf_thr) and (cos >= cos_thr) and (edits <= edit_thr)
    return rf, cos, edits, ok

```

Base/local pair-build for activation patching

Base/local pairs should share the same label and differ primarily in local grounding. Vuk'uzenzele provides clear local grounding cues (province mentions, services distributed across provinces). ³⁴

Operationally:

- `pair_role="base"`: remove SA-specific place names/institutions and replace with generic placeholders ("the local office", "the department").
- `pair_role="local"`: inject one local marker (place/province/institution acronym) into the same slot.

Pair-build scaffold:

```
# src/pair_build.py
from dataclasses import dataclass
from typing import Optional

@dataclass(frozen=True)
class Pair:
    pair_id: str
    base_text: str
    local_text: str

def make_base_local_pair(text: str, local_marker: str, generic_marker: str) -> Pair:
    # If text already contains local marker, produce base by replacing it.
    if local_marker in text:
        base = text.replace(local_marker, generic_marker)
        local = text
    else:
        base = text
        # inject local marker minimally (append locative phrase if safe)
        local = text + f" {local_marker}"
    return Pair(pair_id="", base_text=base, local_text=local)
```

In practice, you should not always append; you should inject via mined templates (from your corpora) such as "at the [institution] in [place]" patterns, but the above shows the mechanics.

Balancing, stratified group splitting, QC probes, and export

Family-based grouping

Define an internal `family_id` = one proposition family representing:

- valid/base
- valid/local
- invalid/base
- invalid/local

Never split families across train/dev/test.

Stratification axes

Stratify on a “stratum key” that compresses your balancing constraints:

- language
- locality (`is_local_task`)
- institution type (internal)
- source (internal)
- perturb type (internal, for invalid examples)

Stratified group split implementation

`StratifiedGroupKFold` returns stratified folds with non-overlapping groups. 35

```
# src/split.py
import numpy as np
import pandas as pd
from sklearn.model_selection import StratifiedGroupKFold

def assign_splits(df: pd.DataFrame, group_col: str, strata_col: str, seed:
int=42) -> pd.DataFrame:
    sgkf = StratifiedGroupKFold(n_splits=20, shuffle=True, random_state=seed)
    groups = df[group_col].to_numpy()
    strata = df[strata_col].to_numpy()

    fold_id = np.full(len(df), -1, dtype=int)
    for k, (_, test_idx) in enumerate(sgkf.split(np.zeros(len(df)), strata,
groups=groups)):
        fold_id[test_idx] = k
    df = df.copy()
    df["fold"] = fold_id

    # 70/15/15 split via folds: first 14 folds train, next 3 dev, last 3 test
    df["split"] = "train"
    df.loc[df["fold"].isin([14,15,16]), "split"] = "dev"
    df.loc[df["fold"].isin([17,18,19]), "split"] = "test"
    return df.drop(columns=["fold"])
```

Leakage detectors (pass/fail thresholds)

Use weak baselines to detect shortcut learnability:

Probe	Features	Pass threshold
Source-only → label	one-hot <code>source</code>	AUC ≤ 0.60

Probe	Features	Pass threshold
Style → label	length, digit ratio, uppercase ratio, punctuation counts	$AUC \leq 0.60$
N-gram → label	TF-IDF word/char n-grams	$AUC < 0.75$ (higher implies lexical shortcuts)

Use scikit-learn logistic regression. 36

```
# src/qc_leakage.py
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import roc_auc_score
from sklearn.feature_extraction.text import TfidfVectorizer

def auc_probe_ngram(texts, y):
    vec = TfidfVectorizer(ngram_range=(1,3), min_df=3, max_df=0.9)
    X = vec.fit_transform(texts)
    y = np.asarray(y, dtype=int)
    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

    aucs = []
    for tr, te in cv.split(X, y):
        clf = LogisticRegression(max_iter=2000, n_jobs=-1)
        clf.fit(X[tr], y[tr])
        p = clf.predict_proba(X[te])[:, 1]
        aucs.append(roc_auc_score(y[te], p))
    return float(np.mean(aucs)), float(np.std(aucs))
```

Export exact schema

```
# src/export.py
import pandas as pd

REQUIRED =
["id", "text", "label", "language", "is_local_task", "pair_id", "pair_role", "split"]

def export_dataset(df: pd.DataFrame, out_csv: str) -> None:
    missing = [c for c in REQUIRED if c not in df.columns]
    if missing:
        raise ValueError(f"Missing required cols: {missing}")
    df[REQUIRED].to_csv(out_csv, index=False)
```

Annotation, adjudication, gold validation sampling, pilot plan, and cost estimates

Annotation UI and workflow

You need “gold” evaluation labels; a pipeline-only dataset is “silver.” A practical approach is to annotate only the gold sets (200 and 600), not the entire dataset.

For deployment, doccano is an open-source annotation tool supporting text classification and collaborative labeling. ³⁷

Recommended annotation fields (stored in a separate annotations file, not necessarily in your published dataset):

- `id` (dataset id)
- `text`
- `language`
- `pair_id`, `pair_role` (for annotator context)
- `label` (VALID/INVALID/UNCERTAIN)
- `error_type` if invalid (institution mismatch / service mismatch / document mismatch / role mismatch / locality mismatch / needs context / other)
- `confidence` (1-5)
- `short_rationale` (required for INVALID or UNCERTAIN)

Calibration task: - 40 items per annotator cohort, balanced across languages and perturbation types, reviewed in a calibration meeting before full gold annotation.

Adjudication: - Two annotators per item; adjudicate disagreements or any UNCERTAIN label, or confidence gap ≥ 2 .

Krippendorff's alpha reporting

The `krippendorff` Python package provides `krippendorff.alpha(reliability_data=...)` and includes example usage in its PyPI description. ²⁶

```
# src/metrics.py
import numpy as np
import krippendorff

LABEL_MAP = {"VALID": 1, "INVALID": 0, "UNCERTAIN": 2}

def krippendorff_alpha_from_long(df, item_col="id", rater_col="rater",
label_col="label"):
    items = df[item_col].unique().tolist()
    raters = df[rater_col].unique().tolist()
```

```

mat = np.full((len(raters), len(items)), np.nan)
item_index = {it:i for i,it in enumerate(items)}
rater_index = {r:i for i,r in enumerate(raters)}

for _, row in df.iterrows():
    i = rater_index[row[rater_col]]
    j = item_index[row[item_col]]
    mat[i, j] = LABEL_MAP[row[label_col]]

return float(krippendorff.alpha(reliability_data=mat,
level_of_measurement="nominal"))

```

Pilot generation plan (240 rows)

Generate 60 families × 4 rows per family (valid/invalid × base/local). Keep the pilot balanced across:

- languages: at minimum `eng`, `zul`, and `afr`, plus at least a few from other official languages to test extraction viability
- sources: 30 families Autshumato, 30 families Vuk'uzenzele
- perturbation types: ensure representation of institution, service, document, role, and (carefully) locality mismatches

Human review checklist per family:

- All 4 items are self-contained without surrounding context
- Base vs local differs mainly by local grounding, not structure
- Invalid is institutionally wrong but not absurd
- No lexical giveaway (e.g., one token always indicates invalid)
- For non-English sentences: no obvious PDF corruption artifacts

200-sample gold validation set generation (sampling code)

This “gold200” is a fast, repeatable evaluation subset you can validate early before scaling.

Sampling principles:

- sample at family-level to preserve structures
- stratify by language and label balance
- oversample “hard negative” perturbation types

```

# src/sample_gold.py
import pandas as pd
import numpy as np

def sample_gold200(examples: pd.DataFrame, family_col="family_id", seed=42) ->
pd.DataFrame:

```

```

rng = np.random.default_rng(seed)

# Work at family level; each family has 4 rows
fam = (examples
        .groupby(family_col)
        .agg(language=("language", "first"),
              label=("label", "mean"),          # should be 0.5 across family
              is_local_task=("is_local_task", "max"),
              n=("id", "count"))
        .reset_index())

# Filter to complete families
fam = fam[fam["n"] == 4].copy()

# Stratify by language; allocate families proportional but ensure at least 1 per
# language present
langs = fam["language"].value_counts()
# choose 50 families => 200 rows
target_families = 50

selected_fams = []
for lang, cnt in langs.items():
    k = max(1, int(round(target_families * (cnt / len(fam)))))
    pool = fam[fam["language"] == lang][family_col].tolist()
    if len(pool) <= k:
        chosen = pool
    else:
        chosen = rng.choice(pool, size=k, replace=False).tolist()
    selected_fams.extend(chosen)

# Trim/expand to exactly target_families
selected_fams = list(dict.fromkeys(selected_fams)) # dedup keep order
if len(selected_fams) > target_families:
    selected_fams = selected_fams[:target_families]
elif len(selected_fams) < target_families:
    remaining = [f for f in fam[family_col].tolist() if f not in
set(selected_fams)]
    add = rng.choice(remaining, size=(target_families-len(selected_fams)),
replace=False).tolist()
    selected_fams.extend(add)

gold = examples[examples[family_col].isin(selected_fams)].copy()
return gold

```

Export `gold200.csv` using your standard `export_dataset()` to keep schema consistent.

Estimated effort and cost to gold a 600-item test set

Assumptions:

- 2 annotators per item + adjudication
- average per-item time:
 - English/Afrikaans: 0.8–1.2 minutes
 - other SA languages: 1.0–1.8 minutes (more time for morphology and OCR artifacts)
- adjudication rate: 15–30%

Estimated person-hours:

Component	Low	High
Primary labeling (600×2)	16 h	36 h
Adjudication	2 h	8 h
Calibration + guideline refinement	4 h	8 h
QA sampling + error analysis	4 h	10 h
Total	26 h	62 h

Costs depend on your annotation pay rates. A practical budgeting method is to multiply total hours by fully loaded hourly cost per bilingual annotator and adjudicator, and to plan recruiting/coordination overhead separately.

```
timeline
  title Practical implementation timeline (local run, no downloads)
  2026-04-06 : Finalize language profiles + licensing plan + schema lock
  2026-04-08 : Implement ingest + normalize + segment (Autshumato +
Vuk'uzenzele)
  2026-04-11 : Implement lexicon matching + relation checks + self-containment
filters
  2026-04-14 : Implement dedup (exact + MinHash + embeddings)
  2026-04-16 : Implement typed perturbations + minimality scoring + base/local
pairing
  2026-04-18 : Implement QC leakage probes + split logic + export
  2026-04-20 : Generate 240-row pilot + human checklist review
  2026-04-24 : Iterate lexicons/thresholds to reduce leakage + boost
plausibility
  2026-04-27 : Generate silver dataset + sample gold200 and gold600
  2026-05-03 : Gold annotation + adjudication + alpha reporting
```

Prioritized next steps

First, implement the pipeline end-to-end for **three languages**: English (spaCy), isiZulu (fallback heuristics), Afrikaans (UD-informed parsing when feasible), because these provide a strong “tooling spectrum” and reveal extraction and perturbation failure modes quickly. Second, iterate lexicons and compatibility matrices using Autshumato alignments and corpus attestation rules (Autshumato aligns English with all other official written SA languages, making it the best lexicon induction substrate). ⁵ Third, treat the leakage probes as acceptance gates: do not scale until n-gram/style/source probes drop toward chance. Fourth, run gold200 early—if alpha is low or adjudication rate is high, fix guidelines and perturbation design before investing in a 600-item gold test set.

Key official/tool references (for your docs)

- Autshumato data description (sentence-level, aligned with English, government domain) ¹
- Autshumato paper (coverage across SA official languages; sourcing from government translations/publications/web crawl) ³⁸
- SADiLaR Autshumato metadata (alignment, sentence segmentation, licensing) ³⁹
- Vuk’uzenzele archives and multilingual edition pages with “Translations” ²⁹
- GCIS statement that Vuk’uzenzele is published twice monthly ⁴⁰
- pdfplumber features and constraints (built on pdfminer.six; works best on machine-generated PDFs) ¹³
- SentencePiece (raw-sentence training; language-independent) ¹⁵
- Sentence-Transformers (embedding framework) ²¹
- RapidFuzz (fast string similarity; MIT licensed) ²³
- datasketch MinHash LSH ²²
- fastText language ID models (176 languages) ²⁵
- scikit-learn StratifiedGroupKFold and LogisticRegression ²⁴
- NLLB translation docs/paper for lexicon candidate generation ³²
- krippendorff alpha package usage ²⁶
- doccano annotation tool ²⁷

¹ ¹⁰ ²⁸ Data – The Autshumato Project

https://autshumato.nwu.ac.za/data/?utm_source=chatgpt.com

² ⁹ ²⁹ Archives - Vuk’uzenzele

https://www.vukuzenzele.gov.za/archives?utm_source=chatgpt.com

³ Use of Official Languages Act: Language policy: Legal Aid South Africa

https://www.gov.za/sites/default/files/gcis_document/201703/40733gen244.pdf?utm_source=chatgpt.com

⁴ The NA Approves South African Sign Language as the 12th Official ...

https://www.parliament.gov.za/press-releases/na-approves-south-african-sign-language-12th-official-language?utm_source=chatgpt.com

⁵ ³⁸ Datasets for South African Languages: Bilingual Aligned and Monolingual ...

https://openhumanitiesdata.metajnl.com/articles/372/files/68cd2e418c690.pdf?utm_source=chatgpt.com

⁶ ³⁹ Autshumato English-Afrikaans Parallel Corpora

https://repo.sadilar.org/items/b8098786-0fe8-4067-8b1f-bb2652843bd6/full?utm_source=chatgpt.com

- 7 Autshumato English-isiZulu Parallel Corpora - repo.sadilar.org
https://repo.sadilar.org/items/0ed26796-7a67-4935-ba57-d3f10e59ba21?utm_source=chatgpt.com
- 8 40 Products and Platforms | Government Communication and ... - GCIS
https://www.gcis.gov.za/content/about-us/chief-directorates/products-platforms?utm_source=chatgpt.com
- 11 Sentencizer · spaCy API Documentation
https://spacy.io/api/sentencizer/?utm_source=chatgpt.com
- 12 UD Afrikaans AfriBooms - Universal Dependencies
https://universaldependencies.org/treebanks/af_afribooms/index.html?utm_source=chatgpt.com
- 13 17 pdfplumber · PyPI
https://pypi.org/project/pdfplumber/?utm_source=chatgpt.com
- 14 18 GitHub - pdfminer/pdfminer.six: Community maintained fork of pdfminer ...
https://github.com/pdfminer/pdfminer.six?utm_source=chatgpt.com
- 15 GitHub - google/sentencepiece: Unsupervised text tokenizer for Neural ...
https://github.com/google/sentencepiece?utm_source=chatgpt.com
- 16 Reading and Writing the Apache Parquet Format - Apache Arrow
https://arrow.apache.org/docs/python/parquet.html?utm_source=chatgpt.com
- 19 Available Models & Languages - Stanza
https://stanfordnlp.github.io/stanza/available_models.html
- 20 GitHub - hplt-project/sacremoses: Python port of Moses tokenizer ...
https://github.com/hplt-project/sacremoses?utm_source=chatgpt.com
- 21 SentenceTransformers Documentation — Sentence Transformers documentation
https://www.sbert.net/?utm_source=chatgpt.com
- 22 GitHub - ekzhu/datasketch: MinHash, LSH, LSH Forest, Weighted MinHash ...
https://github.com/ekzhu/datasketch?utm_source=chatgpt.com
- 23 GitHub - rapidfuzz/RapidFuzz: Rapid fuzzy string matching in Python ...
https://github.com/rapidfuzz/RapidFuzz?utm_source=chatgpt.com
- 24 35 StratifiedGroupKFold — scikit-learn 1.8.0 documentation
https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.StratifiedGroupKFold.html?utm_source=chatgpt.com
- 25 Language identification - fastText
https://fasttext.cc/docs/en/language-identification.html?utm_source=chatgpt.com
- 26 krippendorff · PyPI
https://pypi.org/project/krippendorff/?utm_source=chatgpt.com
- 27 37 GitHub - doccano/doccano: Open source annotation tool for machine ...
https://github.com/doccano/doccano?utm_source=chatgpt.com
- 30 Thusong Service Centres Directory - South African Government
https://www.gov.za/about-government/contact-directory/thusong?utm_source=chatgpt.com
- 31 MULTILINGUAL ARTS AND CULTURE ENGLISH – AFRIKAANS – ISIZU
https://www.dsac.gov.za/sites/default/files/2024-10/Multilingual%20Arts%20and%20Culture%20Intermediate%20phase-terminology.pdf?utm_source=chatgpt.com

32 33 NLLB - Hugging Face

https://huggingface.co/docs/transformers/v4.49.0/en/model_doc/nllb?utm_source=chatgpt.com

34 Where can I find the print newspaper - Vuk'uzenzele

https://www.vukuzenzele.gov.za/where-can-i-find-print-newspaper?utm_source=chatgpt.com

36 LogisticRegression — scikit-learn 1.8.0 documentation

https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html?utm_source=chatgpt.com